9.5 Two-Stream Inflated 3D CNNs (I3D)

- some of the major differences in previous video architectures are whether the convolutional and layers operators use 2D (image-based) or 3D (video-based) kernels; whether the input to the network is just an RGB video or it also includes pre-computed optical flow; and, in the case of 2D CNNs, how information is propagated across frames, which can be done either using temporally-recurrent layers such as LSTMs, or feature aggregation over time
- this paper compares and studies a subset of models that span most of this space
- among 2D CNNs methods, CNNs with LSTMs on top are considered, and two-stream networks with two different types of stream fusion
- a 3D CNN, C3D, is also considered

9.5 Two-Stream Inflated 3D CNNs (I3D)

- as the main technical contribution, the paper introduces Two-Stream Inflated 3D CNNs (I3D)
- due to the high-dimensionality of their parameterization, and the lack of labeled video data, previous 3D CNNs have been relatively shallow (up to 8 layers)
- the paper makes the observation that very deep image classification networks, such as Inception, VGG-16, and ResNet, can be trivially inflated into spatio-temporal feature extractors, and that their pre-trained weights provide a valuable initialization
- a two-stream configuration is also found to still be useful

9.5 Two-Stream Inflated 3D CNNs (I3D)

- a graphical overview of the five types of architectures that are evaluated is shown in Figure 11
- many of these models (all but C3D) have an ImageNet pre-trained model as a subcomponent
- the experimental strategy assumes a common ImageNet pre-trained image classification architecture as backbone, and, for this, Inception-V1 with batch normalization is chosen, which is morphed in different ways
- the expectation is that, with this backbone in common, we will be able to tease apart those changes that benefit action classification the most

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **CNN+LSTM**: the high performance of image classification networks makes it appealing to try to reuse them with as minimal change as possible for video
- this can be achieved by using them to extract features independently from each frame, then pooling their predictions across the whole video
- but while convenient in practice, it has the issue of entirely ignoring temporal structure (e.g., models can't potentially distinguish opening from closing a door)

9.5 Two-Stream Inflated 3D CNNs (I3D)

- in theory, a more satisfying approach is to add a recurrent layer to the model, such as an LSTM, which can encode state and capture temporal ordering and long-range dependencies
- an LSTM layer with batch normalization is positioned after the last average pooling layer of Inception-V1, with 512 hidden units
- a fully-connected layer is added on top for the classifier
- the model is trained using cross-entropy losses on the outputs at all time steps
- during testing, only the output on the last frame is considered
- input video frames are subsampled by keeping one out of every 5, from an original 25 frames-per-second stream

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **3D CNNs**: 3D CNNs seem like a natural approach to video modeling, and are just like standard convolutional networks, but with spatio-temporal filters
- they have a very important characteristic: they directly create hierarchical representations of spatio-temporal data
- one issue with these models is that they have many more parameters than 2D CNNs because of the additional kernel dimension, and this makes them harder to train
- also, they seem to preclude the benefits of ImageNet pre-training, and consequently previous work has defined relatively shallow custom architectures and trained them from scratch
- results on benchmarks have shown promise, but have not been competitive with state-of-the-art

9.5 Two-Stream Inflated 3D CNNs (I3D)

- for the paper, a small variation of C3D was implemented, which has 8 convolutional layers, 5 pooling layers, and 2 fully-connected layers at the top
- the inputs to the model are short 16-frame clips with 112×112-pixel crops, as in the original implementation
- differently from the original paper, batch normalization is used after all convolutional and fully-connected layers
- another difference to the original model is in the first pooling layer, a temporal stride of 2 is used instead of 1, which reduces the memory footprint and allows for bigger batches – this was important for batch normalization (especially after the fully-connected layers, where there is no weight tying)

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two-Stream Networks**: LSTMs on features from the last layers of CNNs can model high-level variation, but may not be able to capture fine low-level motion, which is critical in many cases
- it is also expensive to train, as it requires unrolling the network through multiple frames for backpropagation-through-time
- a different, very practical approach, models short temporal snapshots of videos by averaging the predictions from a single RGB frame and a stack of 10 externally computed optical flow frames, after passing them through two replicas of an ImageNet pre-trained CNN
- the flow stream has an adapted input convolutional layer with twice as many input channels as flow frames (because flow has two channels, horizontal and vertical), and at test time multiple snapshots are sampled from the video and the action prediction is averaged
- this was shown to get very high performance on existing benchmarks, while being very efficient to train and test

9.5 Two-Stream Inflated 3D CNNs (I3D)

- an extension to this architecture fuses the spatial and flow streams after the last network convolutional layer, showing some improvement in performance, while requiring less test-time augmentation (snapshot sampling)
- the current implementation follows this model approximately, using Inception-V1
- the inputs to the network are 5 consecutive RGB frames sampled 10 frames apart, as well as the corresponding optical flow snippets
- the spatial and motion features before the last average pooling layer of Inception-V1 ($5 \times 7 \times 7$ feature grids, corresponding to time, x and y dimensions) are passed through a $3 \times 3 \times 3$ 3D convolutional layer with 512 output channels, followed by a $3 \times 3 \times 3$ 3D max-pooling layer and through a final fully-connected layer
- the weights of these new layers are randomly initialized
- both models, the original two-stream and the 3D fused version, are trained end-to-end (including the two-stream averaging process in the original model)

9.5 Two-Stream Inflated 3D CNNs (I3D)

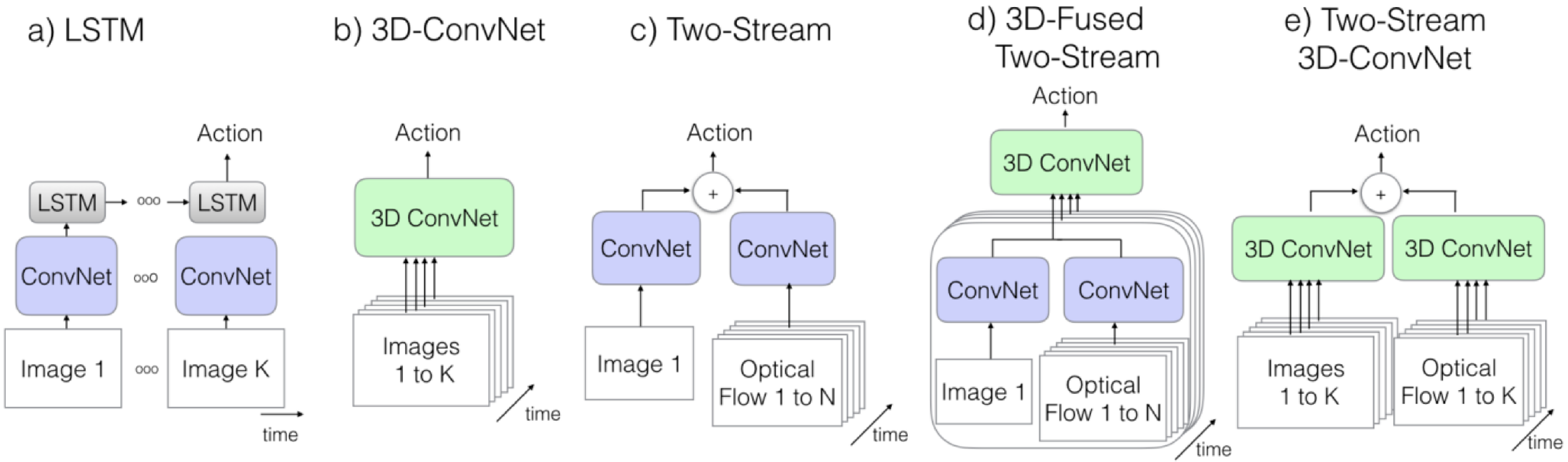


Figure 11: Video architectures considered in the paper. *K* stands for the total number of frames in a video, whereas *N* stands for a subset of neighboring frames of the video.

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two-Stream Inflated 3D CNNs**: with this architecture, it is shown how 3D CNNs can benefit from ImageNet 2D CNN designs and, optionally, from their learned parameters
- a two-stream configuration is also adopted here – it can be shown that, while 3D CNNs can directly learn about temporal patterns from an RGB stream, their performance can still be greatly improved by including an optical flow stream

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Inflating 2D CNNs into 3D**: a number of very successful image classification architectures have been developed over the years, in part through painstaking trial and error
- instead of repeating the process for spatio-temporal models, the paper proposes to simply convert successful image (2D) classification models into 3D CNNs
- this can be done by starting with a 2D architecture, and inflating all the filters and pooling kernels – endowing them with an additional temporal dimension
- filters are typically square and we just make them cubic – $N \times N$ filters become $N \times N \times N$

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Bootstrapping 3D filters from 2D Filters**: besides the architecture, one may also want to bootstrap parameters from the pre-trained ImageNet models
- to do this, we observe that an image can be converted into a (boring) video by copying it repeatedly into a video sequence
- the 3D models can then be implicitly pre-trained on ImageNet, by satisfying what is called the boring-video fixed point: the pooled activations on a boring video should be the same as on the original single-image input
- this can be achieved, thanks to linearity, by repeating the weights of the 2D filters N times along the time dimension, and rescaling them by dividing by N
- this ensures that the convolutional filter response is the same
- since the outputs of convolutional layers for boring videos are constant in time, the outputs of pointwise nonlinearity layers and average and max-pooling layers are the same as for the 2D case, and hence the overall network response respects the boring-video fixed point

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Pacing receptive field growth in space, time, and network depth**: the boring video fixed-point leaves ample freedom on how to inflate pooling operators along the time dimension and on how to set convolutional/pooling temporal stride – these are the primary factors that shape the size of feature receptive fields
- virtually all image models treat the two spatial dimensions (horizontal and vertical) equally – pooling kernels and strides are the same
- this is quite natural and means that features deeper in the networks are equally affected by image locations increasingly far away in both dimensions
- a symmetric receptive field is however not necessarily optimal when also considering time – this should depend on frame rate and image dimensions
- if it grows too quickly in time relative to space, it may conflate edges from different objects, breaking early feature detection, while if it grows too slowly, it may not capture scene dynamics well

9.5 Two-Stream Inflated 3D CNNs (I3D)

- in Inception-V1, the first convolutional layer has stride 2, then there are four max-pooling layers with stride 2 and a 7×7 average-pooling layer preceding the last linear classification layer, besides the max-pooling layers in parallel Inception branches
- the input videos are processed at 25 frames per second; it was found helpful to not perform temporal pooling in the first two max-pooling layers (by using $1 \times 3 \times 3$ kernels and stride 1 in time), while having symmetric kernels and strides in all other max-pooling layers
- the final average pooling layer uses a $2 \times 7 \times 7$ kernel
- the overall architecture is shown in Figure 12
- the model is trained using 64-frame snippets and tested using the whole videos, averaging predictions temporally

9.5 Two-Stream Inflated 3D CNNs (I3D)

- **Two 3D Streams**: while a 3D CNN should be able to learn motion features from RGB inputs directly, it still performs pure feedforward computation, whereas optical flow algorithms are in some sense recurrent (e.g., they perform iterative optimization for the flow fields)
- perhaps because of this lack of recurrence, experimentally it is still found valuable to have a two-stream configuration – shown in Figure 11 e) – with one I3D network trained on RGB inputs, and another on flow inputs which carry optimized, smooth flow information
- the two networks are trained separately and their predictions are averaged at test time

9.5 Two-Stream Inflated 3D CNNs (I3D)

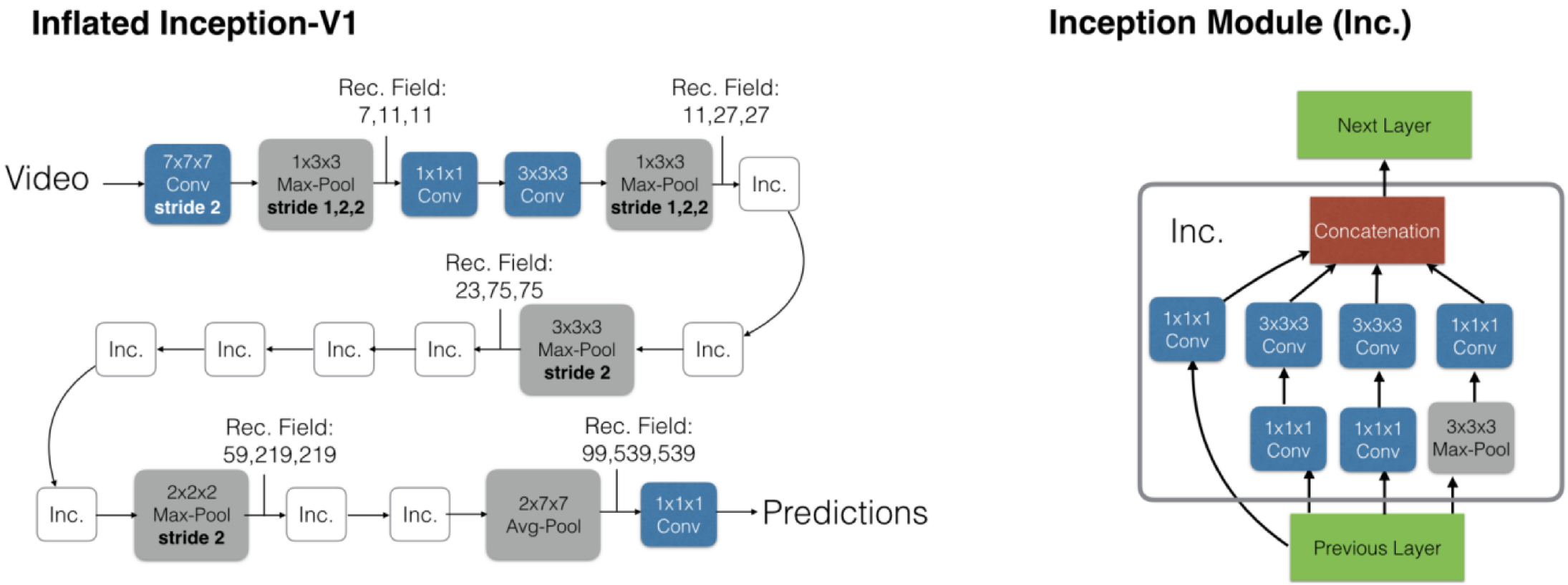


Figure 12: The Inflated Inception-V1 architecture (left) and its detailed inception submodule (right). The strides of convolution and pooling operators are 1 where not specified, and batch normalization layers, ReLUs, and the softmax at the end are not shown. The theoretical sizes of receptive field sizes for a few layers in the network are provided in the format "time,x,y" – the units are frames and pixels. The predictions are obtained convolutionally in time and averaged.